# Slice A — Epic SMART on FHIR Authentication (PKCE)

**Owner: Bassel (Malek support) · Branch: `feature/epic-oauth` · Merge order: last (C → B → A)**

Goal: let a user tap "Connect to Epic (sandbox)," complete the SMART login in the system browser, and return to the app with an access token stored securely — so Slice B can read FHIR. No real PHI; sandbox only.

---

## 1. Design exploration

### Decision 1 — auth library

| Option | Pros | Cons | Verdict |
|---|---|---|---|
| `expo-auth-session` | Handles redirect, PKCE helpers, works in Expo Go via proxy; matches **ADR-002** | Some SMART params set manually | **Chosen** |
| Hand-rolled `WebBrowser` + manual code exchange | Full control | Reimplements PKCE, state, redirect parsing; more to test | Only if a SMART quirk forces it |
| `react-native-app-auth` | Mature OAuth | Needs a config plugin / dev build; not Expo-Go friendly | Rejected for P1 (we target Expo Go SDK 55) |

### Decision 2 — public vs confidential client (open question #7)

| App type | Token exchange happens | Client secret | What we build |
|---|---|---|---|
| **Public (PKCE)** — assume this | In the mobile app | None | `epicAuth.ts` does the exchange (this doc) |
| **Confidential** | In a Supabase Edge Function | `EPIC_CLIENT_SECRET` (server only) | Add `supabase/functions/epic-token`; app sends the `code` + `verifier`, function returns tokens (see §6) |

We build for **public** and isolate the exchange in one function so switching to confidential is a one-function change, not a refactor.

### Decision 3 — token storage

Epic access/refresh tokens go in `expo-secure-store` (Keychain / Keystore), never AsyncStorage, never Supabase, never logged. This is a hard rule from `docs/security-and-scope.md`.

### Decision 4 — where "connected" state lives

An `AuthContext` provider at the app root exposes `{ status, connect(), disconnect(), getAccessToken() }`. This is the state Slice B's `dataSource` checks to decide mock vs. real, and it mirrors the existing

`NavProvider` pattern (a small React context, no new state library).

---

## 2. Files this slice adds

```
apps/mobile/
├── src/services/epicAuth.ts        # discovery, PKCE, authorize, token exchange/refresh
├── src/services/secureTokens.ts    # typed wrapper over expo-secure-store
├── src/features/auth/AuthContext.tsx
└── app/connect-epic.tsx            # the Connect screen (UI by Humayra, logic by Bassel)
```

It also adds one `ScreenName` to `src/navigation/nav.tsx` (`'connect-epic'`) and renders the new screen in the screen switch — the only edit to an existing file.

---

## 3. Environment & Epic registration (do this first)

1. In the Epic on FHIR sandbox (fhir.epic.com), register an app: - **Redirect URI:** `maggie://oauth/callback` (already the app `scheme` in `app.config.ts`). - Note the **client id** (public) → goes in `.env` as `EXPO_PUBLIC_EPIC_CLIENT_ID`. - Scopes: see open question #2; safe default `openid fhirUser launch/patient patient/*.read offline_access`.
2. Confirm `.env` (copied from `.env.example`) has: `EXPO_PUBLIC_EPIC_CLIENT_ID=...` `EXPO_PUBLIC_EPIC_REDIRECT_URI=maggie://oauth/callback` `EXPO_PUBLIC_EPIC_FHIR_BASE_URL=https://fhir.epic.com/interconnect-fhir-oauth/api/FHIR/R4`
3. These are already surfaced on `Constants.expoConfig.extra` via `app.config.ts` — read them from there, exactly as `App.tsx`/screens already read `supabaseUrl`.

> **Guard:** add a runtime check that `epicFhirBaseUrl` contains `fhir.epic.com` (or your sandbox host). Refuse to start the auth flow against an unknown host. This is the "sandbox URL guard" from the security doc.

---

## 4. Implementation walkthrough

### 4.1 `secureTokens.ts` — encrypted storage

```ts
import * as SecureStore from 'expo-secure-store';

const KEY = 'maggie.epic.tokens.v1';

export interface EpicTokens {
  accessToken: string;
  refreshToken?: string;
  patientId?: string;       // SMART "patient" launch context
  expiresAt: number;        // epoch ms
}

export async function saveTokens(t: EpicTokens): Promise<void> {
  await SecureStore.setItemAsync(KEY, JSON.stringify(t));
}
export async function loadTokens(): Promise<EpicTokens | null> {
  const raw = await SecureStore.getItemAsync(KEY);
  return raw ? (JSON.parse(raw) as EpicTokens) : null;
}
export async function clearTokens(): Promise<void> {
  await SecureStore.deleteItemAsync(KEY);
}
```

### 4.2 `epicAuth.ts` — discovery + PKCE + exchange

Key pieces (sketch, not full file):

```
import * as AuthSession from 'expo-auth-session';
import Constants from 'expo-constants';

const extra = Constants.expoConfig?.extra ?? {};
const FHIR_BASE = String(extra.epicFhirBaseUrl ?? '');
const CLIENT_ID = String(extra.epicClientId ?? '');
const REDIRECT = String(extra.epicRedirectUri ?? 'maggie://oauth/callback');

// 1) SMART discovery — find authorize/token endpoints
export async function discover() {
  const res = await fetch(`${FHIR_BASE}/.well-known/smart-configuration`);
  if (!res.ok) throw new Error('SMART discovery failed');
  const cfg = await res.json();
  return { authorizeUrl: cfg.authorization_endpoint, tokenUrl: cfg.token_endpoint };
}

// 2) Build PKCE + request (expo-auth-session generates verifier/challenge)
export async function startAuth() {
  assertSandbox(FHIR_BASE);                       // sandbox URL guard
  const { authorizeUrl, tokenUrl } = await discover();
  const request = new AuthSession.AuthRequest({
    clientId: CLIENT_ID,
    redirectUri: REDIRECT,
    responseType: 'code',
    scopes: ['openid', 'fhirUser', 'launch/patient', 'patient/*.read', 'offline_access'],
    usePKCE: true,                                // S256 via expo-crypto under the hood
    extraParams: { aud: FHIR_BASE },              // SMART requires aud = FHIR base
  });
  const result = await request.promptAsync({ authorizationEndpoint: authorizeUrl });
  if (result.type !== 'success' || !result.params.code) return null;

  // 3) Exchange code → tokens (public client: done here; confidential: see §6)
  const token = await AuthSession.exchangeCodeAsync(
    {
      clientId: CLIENT_ID,
      code: result.params.code,
      redirectUri: REDIRECT,
      extraParams: { code_verifier: request.codeVerifier! },
    },
    { tokenEndpoint: tokenUrl }
  );
  return token; // contains accessToken, refreshToken, expiresIn, and SMART "patient"
}
```

- `assertSandbox()` throws if `FHIR_BASE` is not the sandbox host.
- Epic returns the launch `patient` id alongside the token; capture it into `EpicTokens.patientId` for Slice B's reads.
- Add `refresh()` using `AuthSession.refreshAsync` when `Date.now() > expiresAt`.

### 4.3 `AuthContext.tsx` — app-wide connection state

```
type AuthStatus = 'disconnected' | 'connecting' | 'connected' | 'error';

interface AuthApi {
  status: AuthStatus;
  patientId?: string;
  connect: () => Promise<void>;      // calls startAuth(), saves tokens, sets status
  disconnect: () => Promise<void>;   // clearTokens(), status='disconnected'
  getAccessToken: () => Promise<string | null>; // refreshes if expired
}
```

Wrap `<NavProvider>` with `<AuthProvider>` in `App.tsx`. On mount, `loadTokens()` to restore a prior session (status starts `connected` if a non-expired token exists).

### 4.4 `app/connect-epic.tsx` — the screen

Reuse the existing design system: `ScreenScaffold`, `InfoCard`, `PixelButton`, `MaggieGuideCard`. Maggie `idle`/`wave` pose, an `InfoCard` explaining "you'll sign into the Epic **sandbox**; this is synthetic data," and a `PixelButton` "Connect to Epic ›" calling `auth.connect()`. On success, `navigate('patient-snapshot')`; the snapshot now reads real data through the seam.

Keep the persistent `DemoBadge` — it should now read "sandbox" (real sandbox), still "not medical advice."

---

## 5. Testing on Expo Go / device

- **Redirect URI:** in Expo Go, `maggie://` custom schemes need a dev build or the AuthSession proxy. For SDK-55 Expo Go testing, use `AuthSession.makeRedirectUri({ scheme: 'maggie', path: 'oauth/callback' })` and confirm the value you registered with Epic matches what the device produces (log it once in dev, never the token).
- Test the **happy path** (login → token → patient id), **cancel** (`result.type === 'dismiss'`), and **expired token refresh**.
- Verify with the team's fixed sandbox patient (open question #3) so screenshots are repeatable.

---

## 6. If Epic is a confidential client (open question #7 = confidential)

Only the token exchange changes: 1. Add `supabase/functions/epic-token/index.ts` (Deno). It receives `{ code, codeVerifier, redirectUri }`, adds `EPIC_CLIENT_SECRET` (Supabase secret), POSTs to Epic's token endpoint, returns tokens. 2. In `epicAuth.ts`, replace the `exchangeCodeAsync` block with a `fetch` to that function. 3. The mobile app still never holds the secret. Everything else (storage, context, screen) is identical.

This is why the exchange is isolated in one function — flipping public→confidential is a localized edit.

---

## 7. Guardrails / review checklist (Slice A)

- [ ] No token, code, or full auth URL is ever `console.log`ged outside a dev-only guard.

- [ ] Tokens only in `secureTokens` (SecureStore) — grep the diff for `AsyncStorage`.
- [ ] `assertSandbox()` blocks non-sandbox `EXPO_PUBLIC_EPIC_FHIR_BASE_URL`.
- [ ] No client secret in the app or `EXPO_PUBLIC_*`.
- [ ] `.env` is **not** committed (only `.env.example`).
- [ ] Scopes match what the client/GTA approved (open question #2).

---

## 8. Commit sequence (feature/epic-oauth)

Conventional Commits; push the branch on commit 1 (backup + open a **draft** PR for CI/visibility), push after every green commit thereafter. Full rationale in Doc 04.

| # | Commit message | Contents | Push? |
|---|---|---|---|
| 1 | `chore(auth): add expo-auth-session, crypto, web-browser, secure-store` | `bunx expo install ...`, lockfile | Push → open **draft PR** |
| 2 | `feat(auth): secure token storage wrapper` | `secureTokens.ts` (+ unit test if using jest) | Push |
| 3 | `feat(auth): SMART discovery + PKCE authorize/exchange` | `epicAuth.ts` with `assertSandbox` | Push |
| 4 | `feat(auth): AuthContext for connection state` | `AuthContext.tsx`, wrap in `App.tsx` | Push |
| 5 | `feat(auth): Connect to Epic screen` | `app/connect-epic.tsx`, nav `ScreenName` entry | Push |
| 6 | `test(auth): manual sandbox login verified on device` | notes in PR, optional fixture | Push |
| 7 | `docs(auth): update README + PHI checklist for Epic connect` | docs (Summer) | Push → mark PR **ready for review** |

**When to open the PR:** at commit 1 as a draft (so teammates see direction early and CI runs). **When to request review:** after commit 7, once a real sandbox login works on a device and the guardrail checklist passes. Requires one teammate approval before merging to `develop` (Doc 04).

---

## 9. Definition of done (Slice A)

- Tapping "Connect to Epic" opens the Epic sandbox login and returns to the app.
- A valid access token + launch `patient` id are stored in SecureStore and survive an app restart.
- `AuthContext.status` flips to `connected`; `getAccessToken()` returns a fresh token (auto-refresh).
- No secrets or tokens in logs, git, or `EXPO_PUBLIC_*`.
- Hands Slice B a working `getAccessToken()` and `patientId`.